

LEVEL 15 — NumPy Copy vs View

Line-by-line examples with output

Main idea: A **view** can share the same underlying data with the original array, while a **copy** creates independent data.

1. What is a View?

A view is a new array object that looks at the same underlying data as the original array.

Code:

```
import numpy as np

arr = np.array([10, 20, 30, 40])
view_arr = arr.view()

view_arr[0] = 999

print("Original:", arr)
print("View:", view_arr)
```

Output:

```
Original: [999  20  30  40]
View: [999  20  30  40]
```

Line by line: 1. `import numpy as np` → imports NumPy. 2. `arr = ...` → creates the original array. 3. `arr.view()` → creates a view of `arr`. 4. `view_arr[0] = 999` → changes the shared data. 5. Both arrays show 999 because they share the underlying data.

2. What is a Copy?

A copy creates a completely separate array with its own data.

Code:

```
import numpy as np

arr = np.array([10, 20, 30, 40])
copy_arr = arr.copy()

copy_arr[0] = 999

print("Original:", arr)
print("Copy:", copy_arr)
```

Output:

```
Original: [10 20 30 40]
Copy: [999  20  30  40]
```

Line by line: 1. `arr = ...` → creates the original array. 2. `arr.copy()` → creates independent data. 3. `copy_arr[0] = 999` → changes only the copy. 4. `arr` stays unchanged.

3. Shallow Copy — Simple Meaning

For NumPy arrays, the important practical distinction is usually between a view and an independent copy.

A view shares the underlying array data:

```
arr = np.array([1, 2, 3])
view_arr = arr.view()
```

```
view_arr[1] = 200
```

```
print(arr)
print(view_arr)
```

Output:

```
[ 1 200  3]
[ 1 200  3]
```

Think:

View → "another way to look at the same data".

Important: In general Python, "shallow copy" can have different behavior for nested objects. For NumPy array learning, focus first on **view vs copy**.

4. Deep Copy — Simple Meaning

An independent NumPy copy is made with `copy()`.

Code:

```
arr = np.array([10, 20, 30])
deep_copy = arr.copy()
```

```
deep_copy[2] = 500
```

```
print(arr)
print(deep_copy)
```

Output:

```
[10 20 30]
[ 10  20 500]
```

Think:

Copy → "make separate data".

For a normal NumPy ndarray, `arr.copy()` makes an independent copy of the array's data.

5. `copy()`

`copy()` creates an independent array.

Code:

```
arr = np.array([10, 20, 30])
new_arr = arr.copy()
```

```
new_arr[0] = 100
```

```
print("arr:", arr)
print("new_arr:", new_arr)
```

Output:

```
arr: [10 20 30]
new_arr: [100 20 30]
```

Key point: Changing `new_arr` does not change `arr`.

6. `view()`

`view()` creates a view that normally shares the same underlying data.

Code:

```
arr = np.array([10, 20, 30])
new_arr = arr.view()

new_arr[0] = 100

print("arr:", arr)
print("new_arr:", new_arr)
```

Output:
arr: [100 20 30]
new_arr: [100 20 30]

Key point: Changing new_arr can change arr because the data is shared.

7. Memory Sharing

Memory sharing means two array objects use the same underlying data.

```
Code:
arr = np.array([10, 20, 30])

view_arr = arr.view()
copy_arr = arr.copy()

print("View:", np.shares_memory(arr, view_arr))
print("Copy:", np.shares_memory(arr, copy_arr))
```

Output:
View: True
Copy: False

True → the arrays share memory. False → the arrays do not share memory.

8. np.shares_memory()

np.shares_memory() checks whether two arrays actually share memory.

```
Code:
a = np.array([1, 2, 3, 4])

b = a.view()
c = a.copy()

print(np.shares_memory(a, b))
print(np.shares_memory(a, c))
```

Output:
True
False

b is a view, so it shares memory with a. c is a copy, so it does not share memory with a.

9. np.may_share_memory()

np.may_share_memory() checks whether two arrays might share memory.

```
Code:
a = np.array([1, 2, 3, 4, 5])

b = a[1:4]
c = a.copy()
```

```
print(np.may_share_memory(a, b))
print(np.may_share_memory(a, c))
```

Output:
True
False

`a[1:4]` is a slice/view of `a`, so it may share memory. `c` is an independent copy. Important: `may_share_memory()` is a conservative check. It can say True when memory may overlap; it is not the same as proving actual overlap.

10. View vs Copy — Direct Comparison

```
Code:
arr = np.array([10, 20, 30])

view_arr = arr.view()
copy_arr = arr.copy()

view_arr[0] = 100
copy_arr[1] = 200

print("Original:", arr)
print("View:", view_arr)
print("Copy:", copy_arr)
```

Output:
Original: [100 20 30]
View: [100 20 30]
Copy: [10 200 30]

Why? 1. `view_arr[0]` changed `arr` because `view_arr` shares data. 2. `copy_arr[1]` changed only `copy_arr` because it has separate data.

11. Easy Memory Diagram

```
Original array
|
+----- view() -----> shares same data
|
+----- copy() -----> separate data
```

```
View:
arr  ■■■■■■
      ■■■■ same underlying data
view ■■■■■■
```

```
Copy:
arr  ■■■■■■> Data A

copy ■■■■■■> Data B
```

Data A and Data B are separate.

Remember: `view()` = shared data `copy()` = independent data

12. Quick Revision

```
np.array()      → creates an array
arr.view()      → creates a view
```

`arr.copy()` → creates an independent copy
`np.shares_memory()` → checks actual memory sharing
`np.may_share_memory()` → checks whether memory may be shared

Most important:

View → changes can affect the original.

Copy → changes do not affect the original.

One-line memory trick: VIEW = SAME DATA COPY = SEPARATE DATA